

HireLens: Comprehensive Viva Defense Notes & Technical Guide

B.Tech 3rd Year Final Project | AI-Powered Resume Screening & Job Matching

Project Title:	HireLens: AI-Powered Resume Screening, Skill Gap Analysis & Job Matching System
Team Members:	Aryan (Roll No: 28240533) Nitish (Roll No: 28240529)
Degree & Dept:	B.Tech 3rd Year, Computer Science & Engineering (AI & ML)
Institution:	Panipat Institute of Engineering & Technology (PIET)
Live Web App:	https://aryan8182-hirelens-ai-resume-analyzer-app-p2fpg0.streamlit.app/
GitHub Repo:	https://github.com/Aryan8182/Hirelens-ai-resume-analyzer

1. Executive Overview & System Architecture

HireLens is an intelligent, automated Applicant Tracking System (ATS) designed to eliminate manual resume screening bottlenecks. Unlike traditional keyword-based ATS tools that fail when candidates use synonyms or varying terminology, HireLens employs a **Dual-Vector Semantic Matching Engine** leveraging Sentence-BERT (SBERT) dense vector embeddings combined with Term Frequency-Inverse Document Frequency (TF-IDF) and a 7-domain Natural Language Processing (NLP) skill taxonomy.

Key System Pipeline Stages:

- **Stage 1: Multi-Format Parser:** Extracts structured text from PDF (using *pdfplumber* for layout preservation) and DOCX files.
- **Stage 2: Regex Contact Auditor:** Extracts and validates candidate email addresses and 10-13 digit Indian mobile numbers using non-capturing regex patterns (?:...).
- **Stage 3: 7-Domain Skill Taxonomy Engine:** Categorizes extracted skills across Programming, ML/AI, Data Science, Web Dev, Cloud/DevOps, Databases, and Soft Skills.
- **Stage 4: Dual-Vector Matching Engine:** Computes 384-dimensional dense SBERT embeddings (all-MiniLM-L6-v2) and sparse TF-IDF n-gram vectors.
- **Stage 5: Hybrid ATS Score Computation:** Calculates a weighted 0-100% composite score combining semantic similarity, skill coverage, and keyword match.
- **Stage 6: Recommendation & Database Logging:** Evaluates formatting (contact, length, action verbs, section headers) and logs complete audit records to SQLite (ats_history.db).

2. Mathematical Foundations & Technical Mechanics

Core Mathematical Formulations Used in HireLens

1. Sentence-BERT Cosine Similarity:

$$\text{Cosine Sim}(A, B) = (A \cdot B) / (||A|| \times ||B||) = \sum(A_i \times B_i) / [\sqrt{\sum(A_i^2)} \times \sqrt{\sum(B_i^2)}]$$

Where A and B are 384-dimensional normalized dense vectors produced by all-MiniLM-L6-v2.

2. Skill Coverage Ratio:

$$\text{Skill Ratio} = | \text{Candidate Skills} \cap \text{Required Job Skills} | / | \text{Required Job Skills} |$$

3. Composite Hybrid ATS Score Formula:

$$\text{ATS Score} = (0.45 \times \text{SBERT_Sim}) + (0.35 \times \text{Skill_Ratio}) + (0.20 \times \text{TFIDF_Sim})$$

Weights: 45% Semantic Meaning, 35% Skill Category Overlap, 20% Exact Keyword Matching.

Why SBERT Beats Traditional TF-IDF Alone:

Standard TF-IDF matches exact surface-level words. If a Job Description requests 'Deep Learning Neural Networks' and a candidate writes 'PyTorch Artificial Intelligence Models', TF-IDF gives a 0% match score for those terms. SBERT transforms sentences into dense 384-dimensional vector spaces where semantically similar concepts lie close together, returning a high cosine similarity score (~0.82) despite zero word overlap.

Non-Capturing Regex Contact Info Extractor:

In Python's re.finditer() or re.findall(), capturing groups (...) return only the inside tuple fragment, truncating full mobile numbers. HireLens uses non-capturing groups (?:...):

```
MOBILE_PATTERN = r'(?:\+91[\-\s]?)?[6-9]\d{9}'
```

This ensures Indian numbers (+91-9876543210 or 9876543210) are extracted completely without losing prefix digits.

3. 20 Crucial Viva Defense Questions & Master Answers

Study these 20 categorized Q&As; thoroughly to answer any question from external examiners with complete technical confidence.

Q1: What is the core objective of project HireLens?

Answer: HireLens is an AI-powered automated resume screening and candidate ranking system. It solves the limitation of legacy ATS tools by analyzing both exact keyword matches and deep semantic context between candidate resumes and job descriptions using SBERT and NLP skill taxonomies.

Q2: Why did you choose Sentence-BERT (SBERT) over standard BERT or Word2Vec?

Answer: Standard BERT requires feeding both texts simultaneously into a cross-encoder, which is computationally expensive $O(N^2)$ for ranking large resume pools. Word2Vec averages word vectors, losing sentence context. SBERT (using Bi-Encoder architecture) independently computes fixed 384D sentence embeddings, allowing fast sub-second cosine similarity calculations $O(N)$.

Q3: Explain the 3 components of your ATS Scoring Formula and why these specific weights were chosen.

Answer: The formula is: $ATS\ Score = 0.45(SBERT) + 0.35(Skill\ Ratio) + 0.20(TF-IDF)$. 45% SBERT guarantees semantic understanding (conceptual fit), 35% Skill Ratio enforces hard technical prerequisite matching, and 20% TF-IDF ensures exact terminology aligned with recruiter preferences is rewarded without dominating.

Q4: How does HireLens handle different file formats like PDF and DOCX?

Answer: We implemented a unified parser in `resume\_parser.py`. For PDF files, `pdfplumber` extracts text while preserving physical coordinates and column structures. For DOCX files, `python-docx` extracts text paragraphs. Clean plain text is then passed downstream.

Q5: What NLP technique is used to extract skills from raw resume text?

Answer: We built a rule-based and phrase-matching NLP skill extraction engine in `skill\_extractor.py` covering 7 technical domains. It uses regular expressions with word boundary anchors `\b...\b` to accurately detect multi-word skills like 'Machine Learning', 'React.js', or 'CI/CD' while ignoring false positives.

Q6: What is Cosine Similarity and why is it preferred over Euclidean Distance for text vector matching?

Answer: Cosine Similarity measures the cosine of the angle between two vectors in multi-dimensional space, focusing purely on vector orientation rather than magnitude. Euclidean distance is sensitive to text length (a longer resume has higher magnitude). Cosine similarity normalizes length differences, making short and long resumes comparable.

Q7: What specific SBERT pre-trained model are you using and why?

Answer: We use `all-MiniLM-L6-v2` from Sentence-Transformers. It maps sentences into a 384-dimensional dense vector space. It is extremely fast (under 50ms inference time per document), lightweight (~90MB model size), and achieves high performance on semantic textual similarity benchmarks.

Q8: How do you perform Skill Gap Analysis for a candidate?

Answer: The skill engine compares extracted candidate skills against required skills parsed from the job description. The set difference [Required Skills - Candidate Skills] identifies missing skills, which are displayed as actionable recommendations for the candidate.

Q9: How does HireLens prevent false positives in regex skill extraction (e.g. matching 'Java' inside 'JavaScript')?

Answer: We use word boundary regex anchors `\bJava\b` vs `\bJavaScript\b` and tokenize text into lowercase word tokens before evaluating taxonomy rules, ensuring distinct matches for overlapping skill names.

Q10: What is TF-IDF and how does it contribute to the matching score?

Answer: TF-IDF (Term Frequency-Inverse Document Frequency) measures how important a word is to a document relative to a corpus. In HireLens, `TfidfVectorizer(ngram_range=(1,2))` computes bi-gram sparse term frequencies, scoring how well key industry phrases in the JD match the resume.

Q11: Why did you use non-capturing groups (?:...) in your contact extraction regex?

Answer: Standard capturing parentheses `(...)` in Python `re.finditer()` return tuples of only matched subgroups, dropping the main string context. Non-capturing groups `(?:...)` group pattern logic without splitting the match output, returning full 10-13 digit Indian numbers (+91-XXXXX-XXXXX).

Q12: How is audit history persisted in HireLens?

Answer: We use SQLite via `db_manager.py`. Evaluated candidate resumes, scores, role titles, and timestamps are automatically stored in `ats_history.db`. The Streamlit UI provides real-time log viewing and a one-click 'Clear All Audit Logs' button to reset database records.

Q13: What web frameworks did you use for the user interface and REST API?

Answer: We built an interactive web application using Streamlit (`app.py`) featuring Plotly radar charts and candidate leaderboards. For enterprise microservice integration, we created a FastAPI backend (`api.py`) offering RESTful JSON endpoints.

Q14: Explain the role of Plotly radar charts in your Streamlit dashboard.

Answer: The Plotly polar radar chart visually plots candidate skill proficiency across the 7 taxonomy domains against job description requirements, giving recruiters an instant visual comparison of candidate domain strengths.

Q15: How does the system evaluate resume formatting and structure?

Answer: The recommendation engine checks 5 formatting criteria: (1) Contact information presence, (2) Length check (300-1500 words), (3) Key section header detection (Education, Experience, Skills, Projects), (4) Strong action verb density, and (5) Missing skill keyword density.

Q16: What live test data did you evaluate and what were the resulting scores?

Answer: We tested an ML Engineer resume against a Machine Learning Engineer JD (`sample_data/`). The candidate achieved an 87.5% overall ATS score (SBERT Sim: 0.88, Skill Coverage: 85%, TF-IDF: 0.89), correctly triggering a 'Strong Match' classification.

Q17: What is the average execution latency of HireLens per resume?

Answer: End-to-end execution (parsing, SBERT vector encoding, skill gap computation, and formatting audit) completes in <1.8 seconds per resume on standard Intel i5 CPU hardware without requiring GPU acceleration.

Q18: How does HireLens handle candidate batch processing and ranking?

Answer: In the Streamlit Batch Ranker, recruiters can upload multiple resumes simultaneously. The system loops through each file, computes individual scores in parallel streams, and renders a sorted pandas DataFrame leaderboard from highest to lowest score.

Q19: What are the main technical limitations of the current HireLens implementation?

Answer: Current limitations include: (1) Scanned image-only PDFs require OCR (Tesseract) integration, (2) Taxonomy skill list requires manual updates for newly emerging technologies, and (3) Inference is currently monolingual (English only).

Q20: What future enhancements do you plan to implement post-graduation?

Answer: Future work includes: (1) Integrating Tesseract OCR for scanned PDF support, (2) Utilizing vector databases (FAISS / Qdrant) for million-scale instant candidate search, and (3) Implementing automated interview question generation based on detected candidate skill gaps.

4. Defense Day Strategy & Code Cheat Sheet

Follow this structured sequence during your 10-15 minute college viva defense presentation:

- 1. Introduction (1 Min):** State project title, introduce team members (Aryan & Nitish), and state the problem: traditional ATS systems miss qualified candidates due to rigid keyword matching.
- 2. Live Demonstration (3 Mins):** Open <https://aryan8182-hirelens-ai-resume-analyzer-app-p2fpgo.streamlit.app/>. Click 'Load Preset Sample ML Data' in sidebar to run instant screening.
- 3. Dashboard Walkthrough (3 Mins):** Show the 87.5% Match Score, point to the Plotly Skill Radar Chart, highlight missing skill gaps, and demonstrate the SQLite Clear History button.
- 4. Architecture & Formulas (3 Mins):** Open Slide 6 & 7 from HireLens_Presentation.pptx. Explain the 384D SBERT vector space and the hybrid formula $0.45(\text{SBERT}) + 0.35(\text{Skill}) + 0.20(\text{TFIDF})$.
- 5. Code & Q&A; Defense (3-5 Mins):** Keep ats_matcher.py and resume_parser.py ready in VS Code. Confidently answer examiner questions using Section 3 of these notes!

Code File Quick Reference Table:

File Name	Core Function / Purpose	Key Python Libraries
app.py	Main Streamlit UI, Radar chart rendering, batch ranker, UI layout	streamlit, plotly, pandas
ats_matcher.py	SBERT dense encoding, TF-IDF calculation, hybrid score formula	sentence-transformers, sklearn
skill_extractor.py	7-domain skill extraction ontology & skill gap comparison	re, spacy / nltk
resume_parser.py	PDF/DOCX text extraction & non-capturing regex contact auditor	pdfplumber, python-docx, re
db_manager.py	SQLite database creation, audit logging, and clear history logic	sqlite3
api.py	FastAPI microservice endpoints for enterprise REST integrations	fastapi, uvicorn, pydantic